Comparison and SortedSet

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Comparison and SortedSet

Motivating question:

Which string is bigger: "abacus" or "cat"

Which object is bigger: a balloon, or ten nickels?

```python
numbers = [20, 10, 30]
print(sorted(numbers))

numbers = [Int(20), Int(10), Int(30)]
print(sorted(numbers))
```

```
[10, 20, 30]
Traceback (most recent call last):
File "/Users/rasikabhalerao/2025Fall/2100/Lectures/l25/l25.py", line 34, in <module>
print(sorted(numbers))
      ^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'Int' and 'Int'
```

Where we're headed...

```python
class Int:
    def __init__(self, number: int) -> None:
        self.number = number

    def __repr__(self) -> str:
        return f"{self.number}"

    def __lt__(self, other: Int) -> bool:
        if not isinstance(other, Int):
            return NotImplemented
        return self.number < other.number

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Int):
            return False
        return self.number == other.number
```

```python
numbers = [
    Int(20),
    Int(10),
    Int(30)
]
print(sorted(numbers))
```

```
[10, 20, 30]
```

Comparable protocol (and no interface)

- `__eq__(self, other: object) -> bool`: equals `==`
- `__ne__(self, other: object) -> bool`: not equals `!=`
- `__lt__(self, other: object) -> bool`: less than `<`
- `__le__(self, other: object) -> bool`: less than or equal to `<=`
- `__gt__(self, other: object) -> bool`: greater than `>`
- `__ge__(self, other: object) -> bool`: greater than or equal to `>=`

Don't need all six

Common: Implement `__eq__()` and one ordering method like `__lt__()`

Exercise: Let's write a class for a Plant. Plant are bigger if they get more sunlight.

```
class Plant:
    def __init__(self) -> None:
        self.sunlight_hours = 0

    def get_sunlight(self) -> None:
        self.sunlight_hours += 1

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Plant):
            return False
        return self.sunlight_hours == other.sunlight_hours

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Plant):
            return NotImplemented
        return self.sunlight_hours < other.sunlight_hours

plant1 = Plant()
plant2 = Plant()

plant1.get_sunlight()
```

Poll: What goes in the ??? ?

```
class Bouquet:
    """Bouquets are compared by the number of flowers in them"""
    def __init__(self, flowers: list[Flower]):
        self.flowers = flowers

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Bouquet):
            return False
        return ???

    def __gt__(self, other: object) -> bool:
        if not isinstance(other, Bouquet):
            return NotImplemented
        return ???
```

1. `len(self.flowers) == len(other.flowers)` and `len(self.flowers) < len(other.flowers)`
2. `len(self.flowers) == len(other.flowers)` and `len(self.flowers) > len(other.flowers)`
3. `len(self.flowers) != len(other.flowers)` and `len(self.flowers) < len(other.flowers)`
4. `len(self.flowers) != len(other.flowers)` and `len(self.flowers) > len(other.flowers)`

Poll: What does the comparison protocol do in Python?

1. It lets us define the "length" of an object (the number returned by `len()`)
2. It lets us iterate over the sub-parts of an object (using a `for` loop)
3. It lets us define what it means for an object to be "bigger" or "smaller" than another object
4. It lets us map each object to an int that is used as an index in a hash table

Storing stuff in order

(You may need to `pip3.11 install sortedcontainers`)

```
ss = SortedSet([3, 1, 4, 1, 5])
print(ss)  # SortedSet([1, 3, 4, 5])
ss.add(2)
print(ss)  # SortedSet([1, 2, 3, 4, 5])
```

```
sm = SortedDict({2: [1, 2, 3], 1: [0, 0, 0]})
print(sm)  # SortedDict({1: [0, 0, 0], 2: [1, 2, 3]})
```

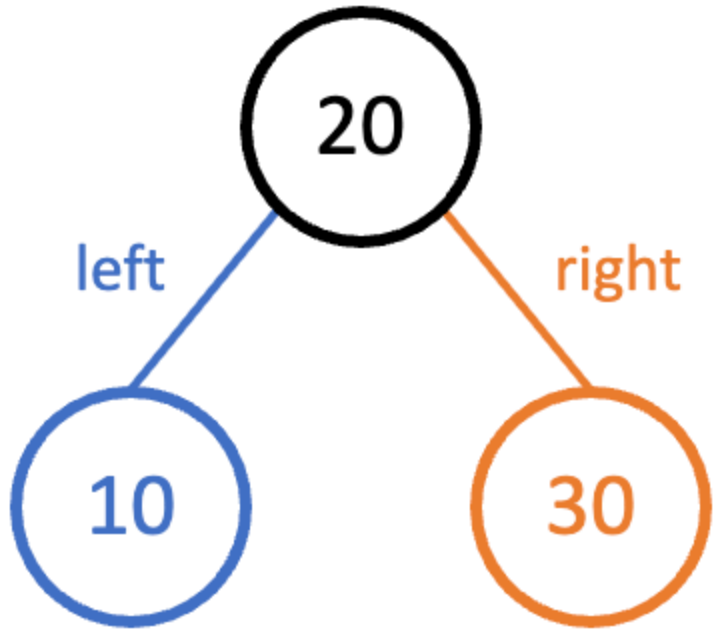
Exercise: let's create a sorted set of `Bouquet`s and iterate over it

Binary Search Trees

Binary Tree: A tree in which each node has at most 2 children

The first and second child of a node are called the left and right child, respectively.

Binary Search Tree: A binary tree in which each node's data is greater than everything in its left subtree and less than everything in its right subtree



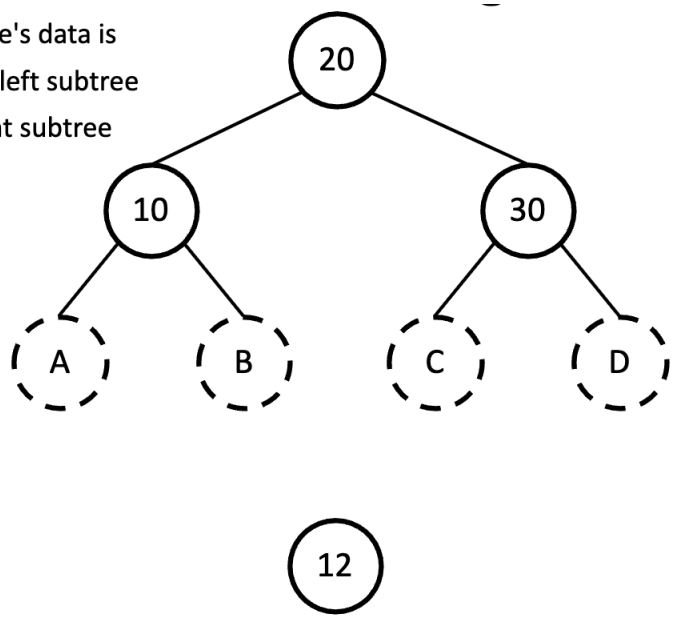
A binary tree in which each node's data is

- greater than everything in its left subtree
- less than everything in its right subtree

Poll: This is a Binary Search Tree. Where should the 12 go?

A binary tree in which each node's data is

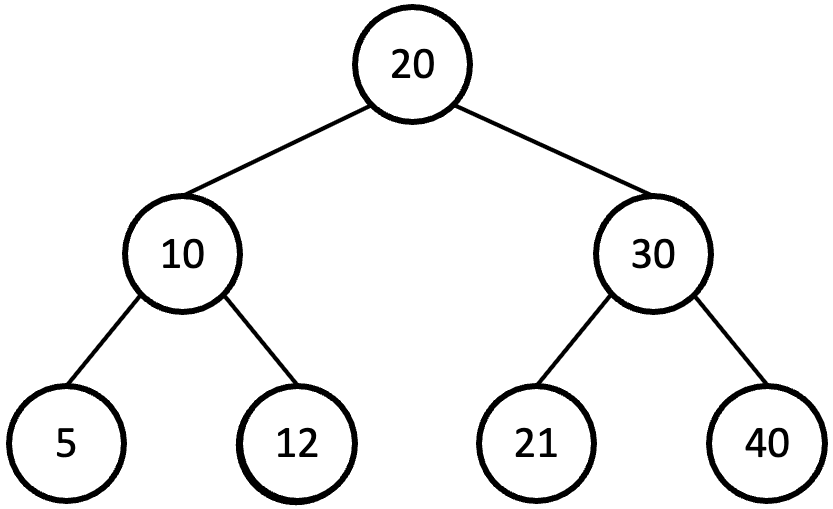
- greater than everything in its left subtree
- less than everything in its right subtree



1. A
2. B
3. C
4. D

What's a Binary Search Tree good for?

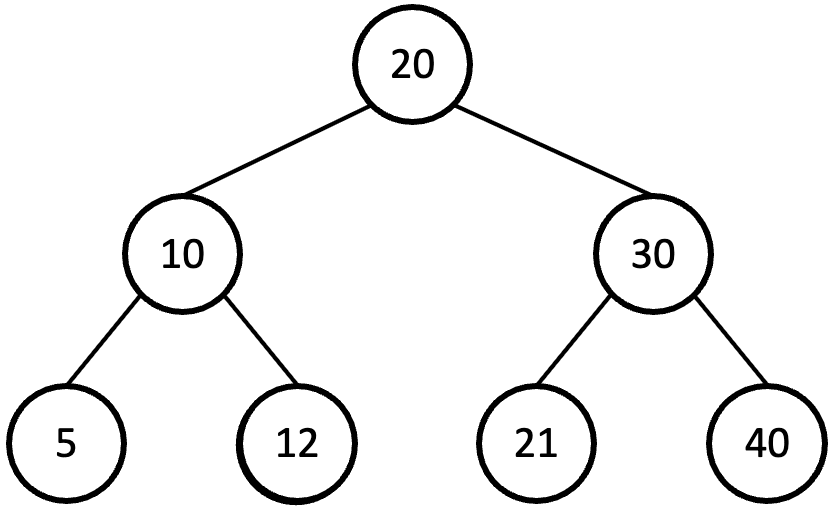
Let's say I want to check for whether the tree contains 12. How do we do that?



What about searching for 11 (with the same BST)?

What's a Binary Search Tree good for?

Let's say I want to check for whether the tree contains 12. How do we do that?



What about searching for 11 (with the same BST)?

Hey, this is more efficient than searching a list!

It's not as efficient as hashing, but it does make a pretty good set.

And it's more useful if you care about the order of things.

`sortedcontainers.SortedSet` stores elements using a Binary Search Tree

(`pip install sortedcontainers`)

```
ss = SortedSet([3, 1, 4, 1, 5])
print(ss)  # SortedSet([1, 3, 4, 5])
ss.add(2)
print(ss)  # SortedSet([1, 2, 3, 4, 5])
```

Corresponding `SortedDict` :

```
sm = SortedDict({2: [1, 2, 3], 1: [0, 0, 0]})
print(sm)  # SortedDict({1: [0, 0, 0], 2: [1, 2, 3]})
```

`set`	`sortedcontainers.SortedSet`
Stored as a hash table (list of lists) Index of each element is calculated using `__hash__()`	Stored as a Binary Search Tree Elements must implement Comparable protocol
Constant time to look up / add / remove	Logarithmic time to look up / add / remove
Use when care more about speed than order	Use when care more about order than speed

Poll: Which are true?

1. For a `SortedDict`, it is constant time to check whether it contains a key
2. For a `SortedDict`, it is constant time to check whether it contains a value
3. `set`s can store things which don't implement the Comparable protocol
4. `set`s can store things which aren't hashable
5. When we iterate over a `set`, the elements will be increasing in size
6. When we iterate over a `SortedSet` the elements will be increasing in size

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?